Efficient Vision-Language-Action Models via Hybrid Diffusion Heads

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—Vision-Language-Action (VLA) models map images, language and robot state to actions, but adapting a pre-trained VLA to a specific task usually means fine-tuning hundreds of millions of parameters. We study a cheaper alternative: the VLA is kept completely frozen and a small generative head is trained on top of it. Our *hybrid diffusion* head is a conditional flow-matching model that predicts a *residual* action chunk. It is conditioned on (i) a pooled semantic feature from the frozen SmolVLA backbone and (ii) the action chunk that the frozen SmolVLA policy itself proposes for the same observation; the executed action is the base action plus the sampled residual. Because the backbone is frozen, its outputs are computed once per demonstration frame and cached, so the 8.5 M-parameter head trains in about two minutes on one GPU. In a first experiment on one LIBERO-10 task (task 3), the residual policy succeeds in 48 of 50 episodes against 43 of 50 for the frozen base policy on the same initial states. This difference is encouraging but not statistically significant at this sample size (exact McNemar $p = 0.125$); evaluation on the remaining LIBERO-10 tasks is in progress.**

***Index Terms*—Vision-Language-Action, Diffusion Models, Flow Matching, Residual Policy Learning, Imitation Learning**

I. Introduction

End-to-end robotic manipulation relies on mapping high-dimensional observations (camera images, robot proprioception) and a natural-language instruction to continuous action sequences. Vision-Language-Action (VLA) models such as RT-2 [1], OpenVLA [2], π_0 [3] and SmolVLA [4] show that pre-trained Vision-Language Models (VLMs) can be turned into capable robot policies.

A pre-trained VLA is rarely perfect on a given task. On LIBERO-10 [5], the public SmolVLA checkpoint used in this work succeeds on roughly 41% of episodes on average, with per-task rates between 0% and 90% (Table II). The standard remedy, fine-tuning the VLA, is expensive and risks degrading what the model already does well. Diffusion policies [6] are a strong alternative for imitation learning, but training one from scratch discards the semantic grounding of the VLA.

We ask whether a small generative head attached to a *frozen* VLA can improve it. Our *hybrid diffusion* approach keeps every SmolVLA weight fixed and trains a lightweight conditional flow-matching head that outputs a correction to the base policy's action chunk. The design rests on three points, each of which we found necessary in practice:

- **The residual is conditioned on the base action it corrects.** The base policy is itself a stochastic sampler, so a residual is only well defined relative to one particular base sample. The head therefore receives that sample as an input.
- **Training and rollout use the same base sampler and the same observation pipeline.** The residual target is computed against the action chunk the base policy would actually execute (same number of integration steps, same rendered observation), not against a cheaper approximation.
- **Frozen-backbone outputs are cached.** The semantic feature and the base action chunks are computed once per demonstration frame, which reduces head training to a few minutes.

II. Related Work

A. Vision-Language-Action Models

VLA models co-train or fine-tune large vision-language models on robot demonstrations. RT-2 [1] and OpenVLA [2] emit discretised action tokens, whereas π_0 [3] and SmolVLA [4] attach a flow-matching action expert that generates continuous action chunks while attending to the VLM's internal representations. SmolVLA, used here, is built on the SmolVLM2-500M backbone.

B. Diffusion and Flow-Matching Policies

Diffusion Policy [6] formulates control as conditional denoising of an action trajectory [7]. Flow matching [8] trains a velocity field that transports noise to data along straight paths and is sampled with a few Euler steps. Such policies represent multi-modal action distributions well, but typically rely on visual encoders without language grounding.

C. Residual Policies

Residual policy learning [9], [10] keeps a base controller fixed and learns an additive correction. It has mostly been studied with reinforcement learning on top of a deterministic controller. Our setting differs in two respects: the correction is learned purely from demonstrations, and the base policy is stochastic, which motivates conditioning the residual model on the sampled base action.

III. METHOD

Fig. 1 gives an overview. At every control step the frozen SmolVLA processes the observation once. That single pass yields a semantic feature and a base action chunk; the trainable head turns these into a residual chunk; and the sum of the two is executed.

A. Frozen Base Policy

The base policy is the public SmolVLA checkpoint fine-tuned on LIBERO [4]. It consists of a SmolVLM2-500M backbone and an action expert trained with flow matching. An observation $o = (I^{\text{agent}}, I^{\text{wrist}}, s, \ell)$ comprises two 256×256 RGB images, the 8-D robot state s (end-effector position, axis-angle orientation and two gripper joint positions) and the instruction ℓ. SmolVLA embeds these as a *prefix* token sequence and runs the backbone over it once, producing per-token hidden states and a key–value (KV) cache. The action expert then integrates a velocity field from Gaussian noise to an action chunk in 10 Euler steps, cross-attending to the cached prefix. We keep the first $H = 16$ steps and the $A = 7$ action dimensions (three translation, three rotation, one gripper) of its output and denote this base chunk $a^b \in \mathbb{R}^{H \times A}$. All actions in this section are in SmolVLA's normalised action space (per-dimension mean and standard deviation of the training data). No SmolVLA parameter receives gradients.

B. Semantic Conditioning Feature

From the same prefix pass we take the backbone's final hidden states $h_{1:L} \in \mathbb{R}^{L \times 960}$ and mean-pool them over the prefix tokens,

$$z = \frac{1}{L} \sum_{i=1}^{L} h_i \in \mathbb{R}^{960}. \tag{1}$$

The prefix is processed exactly as in SmolVLA's own inference path (prefix self-attention only, no action tokens), so z is computed from the representation the action expert was trained against. The pooled feature changes only slightly from frame to frame relative to its mean, so the head standardises it, $\tilde{z} = (z - \mu_z)/\sigma_z$, with per-dimension statistics μ_z, σ_z computed on the training frames and stored in the checkpoint.

C. Residual Target

Let $a^\star \in \mathbb{R}^{H \times A}$ be the demonstrated action chunk starting at the current frame (padded at the end of an episode by repeating the last action). The head models the distribution of the residual

$$r = a^\star - a^b, \qquad a^b \sim \pi_{\text{base}}(\cdot \mid o). \tag{2}$$

The base policy is stochastic: two samples of a^b for the same observation differ noticeably (on task 3 the per-dimension standard deviation of the first action across samples is 0.09–0.59 in normalised units). The correction that maps a^b to $a^\star$ therefore depends on which a^b was drawn, and a head that does not observe a^b can at best learn the average correction. We therefore condition the head on a^b, and we sample a^b with the same 10-step sampler that is used at rollout time so that the training targets match the residuals encountered during execution.

D. Conditional Flow-Matching Head

The head v_θ is trained with conditional flow matching [8]. With $\epsilon \sim \mathcal{N}(0, I)$ and $t \sim \mathcal{U}(0, 1)$,

$$x_t = t\, r + (1 - t)\, \epsilon, \tag{3}$$

$$\mathcal{L}_{\text{flow}} = \mathbb{E} \left\| v_\theta(x_t, t, \tilde{z}, a^b) - (r - \epsilon) \right\|_2^2. \tag{4}$$

At inference a residual is sampled by integrating from $x_0 \sim \mathcal{N}(0, I)$ at $t = 0$ to $t = 1$ with $N = 10$ Euler steps, $x_{t+1/N} = x_t + \frac{1}{N} v_\theta(x_t, t, \tilde{z}, a^b)$, and $\hat{r} = x_1$.

Fig. 2 shows the network. The noisy chunk x_t is flattened to 112 values and projected to a hidden vector $h \in \mathbb{R}^d$ with $d = 512$. The conditioning vector is the sum of three embeddings,

$$e = \phi_t(1000\, t) + \phi_z(\tilde{z}) + \phi_a(a^b), \tag{5}$$

where ϕ_t is a sinusoidal embedding and ϕ_z, ϕ_a are two-layer MLPs. Flow time is multiplied by 1000 before the sinusoidal embedding because the embedding's frequencies are designed for integer diffusion steps and barely resolve $t \in [0, 1]$ otherwise. The trunk is a stack of six residual blocks with FiLM conditioning [11],

$$h \leftarrow h + W_2\, \mathrm{GELU}\big((1 + \gamma(e)) \odot W_1 h + \beta(e)\big), \tag{6}$$

followed by a two-layer MLP that outputs the velocity for the whole chunk. The head has 8.5 M parameters.

E. Training on a Feature Cache

Since the backbone is frozen, everything the head needs from it is a deterministic function of the observation (the feature z) or can be pre-sampled (the base chunk a^b). Training therefore proceeds in two stages (Fig. 1b).

Stage 1: cache. For every frame of every demonstration of the task, the recorded simulator state is restored in the LIBERO simulator and the observation is rendered through the same code path that the evaluation uses (256×256 images rotated by $180°$, the same state vector and the same SmolVLA pre-processor). This removes any mismatch between the stored demonstration images and what the policy sees at rollout. The frozen SmolVLA is run once on each frame and we store z, $K = 4$ independent samples $a_1^b, \ldots, a_K^b$ of the base chunk, and the normalised demonstration chunk $a^\star$. For task 3 this gives 12,434 frames from 50 demonstrations.

Stage 2: head training. Each optimisation step draws a mini-batch of frames, picks one of the K cached base samples per frame uniformly at random, forms the residual of Eq. (2) and minimises Eq. (4). Using several base samples per frame exposes the head to the variability of the base policy without re-running the backbone. As an augmentation, Gaussian noise with standard deviation 0.5 is added to the standardised feature $\tilde{z}$ during training, which reduces over-fitting to the small number of demonstrations. We train with AdamW (learning rate 3×10^{-4} with cosine decay, weight decay 10^{-4}, batch size 256, gradient-norm clipping at 1) for 300 epochs and keep

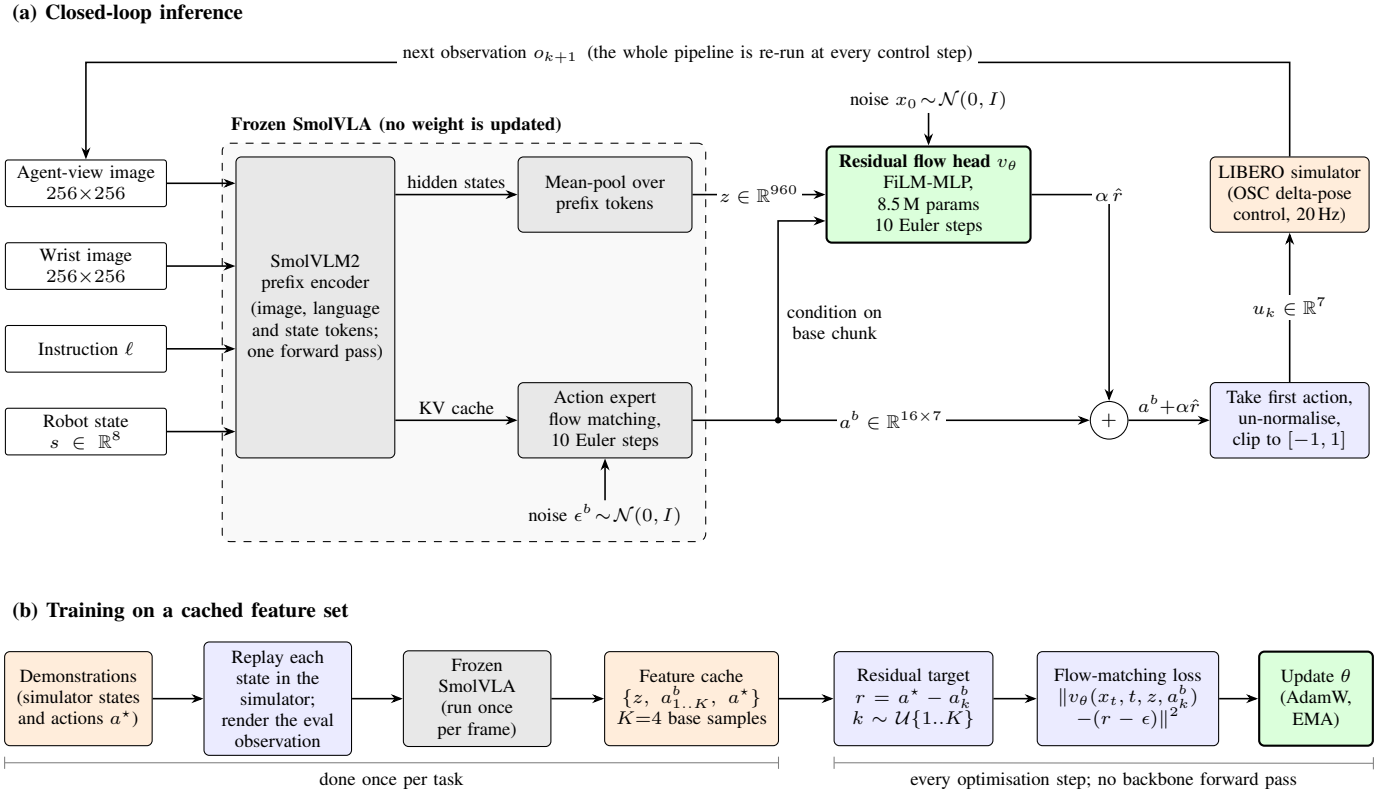


Fig. 1. Overview of the residual hybrid-diffusion policy. **(a)** Inference. The frozen SmolVLA encodes the two camera images, the instruction and the robot state in a single prefix pass. Its hidden states are mean-pooled into the semantic feature z, and its key–value cache drives SmolVLA's own action expert, which samples the base action chunk a^b. The trainable head (green) samples a residual chunk $\hat{r}$ conditioned on both z and a^b. The first action of $a^b + \alpha\hat{r}$ is un-normalised, clipped and executed, and the procedure repeats on the next observation. **(b)** Training. Demonstration states are replayed in the simulator and passed once through the frozen model to build a cache of features, base-action samples and demonstrated actions; the head is then trained on the cache with a flow-matching loss on the residual $a^\star - a^b$. Grey: frozen; green: trainable; orange: data and environment.

an exponential moving average of the weights (decay 0.999), which is the model that is evaluated. Stage 1 takes about 40 minutes for one task on a single laptop GPU and Stage 2 about two minutes.

F. Closed-Loop Inference

At control step k the policy executes

$$u_k = \mathrm{clip}\big(\mathrm{unnorm}\big([\,a^b + \alpha\,\hat{r}\,]_1\big), -1, 1\big), \tag{7}$$

where $[\cdot]_1$ selects the first action of the chunk, unnorm maps back to environment units and α scales the correction ($\alpha = 1$ in our experiments; $\alpha = 0$ recovers the base policy). The base chunk and the residual are produced from the same observation and the same prefix pass, and the residual is conditioned on exactly the base chunk it is added to. A new chunk is generated at every control step, which is also how the base SmolVLA checkpoint is run (it executes one action per predicted chunk). Clipping is applied in environment units, after un-normalisation, because ± 1 in the normalised space corresponds to only one standard deviation of the action distribution.

IV. Experiments

A. Setup

We evaluate on LIBERO-10 [5]. Each task provides 50 demonstrations and 50 benchmark initial states. An episode starts from a benchmark initial state, runs 10 no-op steps to let the scene settle, and then executes the policy with operational-space delta-pose control for at most 520 steps; it counts as a success if the task's goal predicate becomes true. The frozen base policy and the residual policy are evaluated through the same code path on the same initial states, with the random seed of the sampling noise set from the episode index, so the comparison is paired. We report the number of successes, Wilson 95% confidence intervals and an exact McNemar test on the paired outcomes.

Head width, depth and the feature-noise level were selected on five held-out demonstrations of task 3 using the action error defined below, not on rollouts. The head evaluated in simulation was trained on the remaining 45 demonstrations.

B. Offline Action Error

Before running rollouts we measure, on held-out demonstration frames, how close the first action of the chunk is to the demonstrated action with and without the correction (Table I).

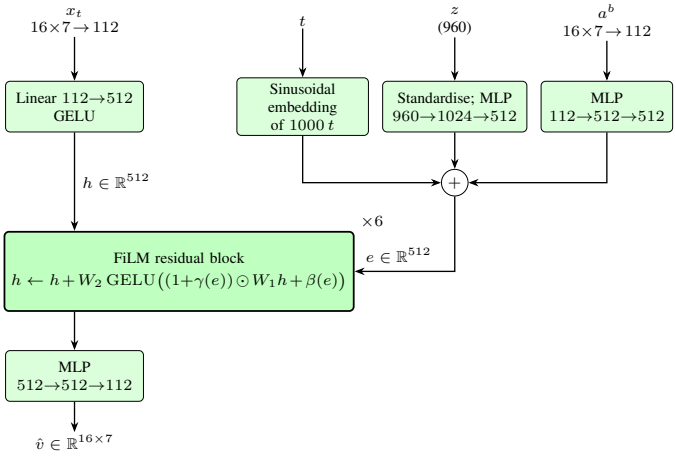


Fig. 2. Architecture of the residual flow head v_θ. The noisy residual chunk is flattened and projected to a 512-D hidden vector h. Flow time, the semantic feature and the base action chunk are embedded separately and summed into one conditioning vector e, which modulates each of six residual blocks through a feature-wise scale $\gamma(e)$ and shift $\beta(e)$. The output is the predicted velocity for all 16 steps of the chunk.

TABLE I
OFFLINE ACTION ERROR ON TASK 3 (MEAN SQUARED ERROR TO THE DEMONSTRATED ACTION, NORMALISED ACTION UNITS, LOWER IS BETTER).

	Base a^b	Base + residual
First action, held-out demos (5)	0.462	0.345
Full 16-step chunk, held-out demos (5)	0.373	0.362
First action, training demos (45)	0.556	0.037

The residual reduces the first-step squared error by about 25% on unseen demonstrations. The gap between training and held-out error is large, which reflects the small data set (45 demonstrations) rather than limited model capacity.

C. Closed-Loop Results

Table II lists closed-loop success. On task 3 (*put the black bowl in the bottom drawer of the cabinet and close it*) the frozen base policy succeeds in 43 of 50 episodes (86%, 95% CI 74–93%) and the residual policy in 48 of 50 (96%, 95% CI 87–99%). Looking at the paired outcomes, the residual policy solves six initial states on which the base policy fails and fails on one that the base policy solves; one initial state is failed by both. With seven discordant pairs the exact McNemar test gives $p = 0.125$, so at this sample size the improvement cannot be distinguished from chance. Successful episodes take a similar number of steps under both policies (256 for the base policy and 268 for the residual policy on average).

D. Limitations of the Current Evidence

The closed-loop result covers a single task, a single training seed for the head and 50 episodes. Task 3 is also one on which the base policy is already strong, which leaves little room for improvement; tasks on which the base policy succeeds less often are more informative and are being evaluated. All 50 benchmark initial states of the task are used, so stronger evidence has to come from additional tasks and training seeds rather than from more episodes on this task. Finally, the head is trained per task.

TABLE II
CLOSED-LOOP SUCCESS ON LIBERO-10. "REFERENCE" IS THE FROZEN BASE POLICY EVALUATED WITH THE STANDARD LEROBOT EVALUATION SCRIPT (3 SEEDS × 10 EPISODES). "BASE" AND "RESIDUAL" ARE EVALUATED WITH OUR HARNESS ON THE 50 BENCHMARK INITIAL STATES OF EACH TASK (SUCCESSES OUT OF 50). ROWS MARKED – HAVE NOT BEEN RUN YET.

Task	Reference (%)	Base	Residual	p (McNemar)
0	16.7	–	–	–
1	36.7	–	–	–
2	40.0	–	–	–
3	83.3	43/50	48/50	0.125
4	0.0	–	–	–
5	90.0	–	–	–
6	70.0	–	–	–
7	10.0	–	–	–
8	13.3	–	–	–
9	46.7	–	–	–
Mean	40.7	–	–	

V. CONCLUSION

[To be completed once the LIBERO-10 evaluation in Table II is finished. The conclusion should state whether a frozen-backbone residual flow head improves the base policy across tasks, and on which kind of task it helps or hurts.]

REFERENCES

[1] A. Brohan *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Proc. Conf. Robot Learning (CoRL)*, 2023.

[2] M. J. Kim *et al.*, "OpenVLA: An open-source vision-language-action model," in *Proc. Conf. Robot Learning (CoRL)*, 2024.

[3] K. Black *et al.*, "π_0: A vision-language-action flow model for general robot control," arXiv:2410.24164, 2024.

[4] M. Shukor *et al.*, "SmolVLA: A vision-language-action model for affordable and efficient robotics," arXiv:2506.01844, 2025.

[5] B. Liu *et al.*, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] C. Chi *et al.*, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Proc. Robotics: Science and Systems (RSS)*, 2023.

[7] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[8] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[9] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, "Residual policy learning," arXiv:1812.06298, 2018.

[10] T. Johannink *et al.*, "Residual reinforcement learning for robot control," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2019.

[11] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. AAAI Conf. Artificial Intelligence*, 2018.